

Language Specific

Interview Cracker question bank

Generated from the Markdown sources in the Interview Cracker repository. Every code snippet in these pages was executed and checked before publication.

C Interview Questions

[Easy] Q. `malloc`, `calloc`, `realloc` - what does each one do, and what do

they initialize?

Answer. `malloc(n)` allocates `n` bytes and returns a suitably aligned pointer, or `NULL` - the bytes are **uninitialized** (indeterminate values). `calloc(n, size)` allocates `n * size` bytes and **zero-fills** them. `realloc(p, n)` resizes the block pointed to by `p` to `n` bytes, moving it if necessary, and returns the (possibly new) pointer - `realloc(NULL, n)` is the same as `malloc(n)`, and `realloc(p, 0)` is implementation-defined.

The follow-ups:

- Check every allocation for `NULL` before using it - the size that made it
- `calloc`'s zero-fill is why it is the right choice for numeric arrays you
- The `realloc` trap that loses memory:
- The allocation is for **bytes**; `malloc(n * sizeof *p)` (not

[Easy] Q. `char *s = "abc";` versus `char t[] = "abc";` - what is different?

Answer. The first points at a string literal (read-only storage, usually in a `.rodata`-style section); the second **copies** the literal into a new array on the stack that you own. So `t[0] = 'X';` is fine, while `s[0] = 'X';` is undefined behaviour - and it is the classic "worked on the old machine, segfaults on the new one" bug, because older toolchains kept literals in writable memory.

Two more consequences interviewers probe:

- `sizeof("abc")` is 4 and `sizeof t` is 4 (both include the NUL), but
- `strlen` needs the NUL; an array like `char three[3] = {'x', 'y', 'z'};` is

[Medium] Q. Why is `sizeof(arr)` different inside the function that created the

array from inside the function that received it?

Answer. Because in C an array parameter is **not** an array - the declaration `void f(int a[10])` is adjusted to `void f(int *a)`. This is called "array-to-pointer decay," and it happens at every function boundary: the callee receives a pointer, so `sizeof a` is the pointer size, and there is no length to recover. (It also means `a[3]` works in both places, because subscripting a pointer and an array is the same operation: `a[3]` is `*(a + 3)`.)

The practical consequences:

- Functions that take arrays must **also** take the length (or a NUL sentinel)
- `sizeof` is still useful on the defining side:
- Variadic `printf` solves the same information-loss problem a different way:

[Medium] Q. What are the `free()` mistakes, and why do people set pointers to NULL after freeing?

Answer. The four: **double free** (`free(p); free(p);` - the allocator's metadata is corrupt, this is UB and a classic crash/exploit primitive), **freeing what you did not allocate** (stack variables, sub-objects of a larger block, or a pointer that was advanced - `free(p + 1)` is UB), **use after free** (reading or writing through the pointer after the block went back to the pool - the memory may already belong to someone else), and **leaks** (losing the only pointer without freeing).

Setting `p = NULL` after `free(p)` does not prevent any of these by itself - it turns use after free and double free from "undefined" into "almost certainly a clean NULL dereference you can find," because `free(NULL)` is a defined no-op while `free(garbage)` is not. The real defenses are: one allocation, one free, on one pointer; short pointer lifetimes; and tools (Valgrind, ASan) that catch all four in the test run rather than in production.

[Hard] Q. What is undefined behaviour, why does the standard allow it, and which ones are the common offenders?

Answer. Undefined behaviour is any program behaviour the standard does not constrain: nothing is required to happen, including "crash," "print garbage," or "work perfectly." The standard gets away with this because compilers are allowed to **optimize under the assumption that UB never happens in the program** - code that contains UB may be transformed as if the UB-producing line could not execute, which is how "harmless" UB becomes a silent logic bug.

The common offenders, and why each is dangerous:

- **Signed integer overflow** (`INT_MAX + 1`). Because it is UB, a compiler
- **Out-of-bounds reads/writes** - including one past the end for "convenience"
- **Dangling pointers** - use after free, or a pointer to a local after return.
- **Type punning through strict aliasing** - reading an `int` through a
- **Uninitialized reads** - the value is indeterminate, and using it is UB.
- **Null dereference** - even `if (!p)` is not safe to rely on if `p`'s value

The interview-grade framing: UB is not "it probably works." It is "the compiler may assume your program is a different, well-defined program." The defenses are the same as for memory bugs - narrow pointer lifetimes, bounds carried as explicit parameters, and sanitizers/Valgrind in CI.

See also `examples/c/strings/string_literal_vs_char_array.c` and `examples/c/pointers/dynamic_int_array.c`.

C++ Interview Questions

[Easy] Q. What is RAII, and why is it the centre of C++ resource management?

Answer. Resource Acquisition Is Initialization: bind a resource (memory, file handle, lock, socket) to an object's **lifetime** - acquire it in the constructor, release it in the destructor. Because destructors run deterministically when the object goes out of scope, "cleanup" stops being a thing you remember to do and becomes a thing the language does for you, on every exit path including exceptions.

The follow-ups:

- This is why C++ has no garbage collector and no `finally`: a `std::lock_guard`
 - The flip side: destructors must be **predictable and cheap-ish**, and an
 - The pattern's limits: RAII owns resources with a clear owner and scope. For
-

[Easy] Q. `unique_ptr`, `shared_ptr`, `weak_ptr` - when is each one right?

Answer. `unique_ptr` is the default: single ownership, zero overhead (same size as a raw pointer), non-copyable but movable. `shared_ptr` is reference-counted shared ownership - use it when you genuinely cannot decide the owner up front. `weak_ptr` observes a `shared_ptr`'d object without extending its life; `lock()` upgrades to a `shared_ptr` or gives you nothing.

The trade-offs that matter in an interview:

- `shared_ptr` costs an atomic increment/decrement on every copy and a second
 - **Cycles:** two `shared_ptr`s pointing at each other never hit zero and
 - A `unique_ptr` can be a member; a `shared_ptr` cycle can hide for months.
-

[Medium] Q. What does "move semantics" actually do, and when does a move happen implicitly?

Answer. A **move** transfers a resource from one object to another instead of copying it: the moved-from object is left valid-but-unspecified (usually empty), and the expensive copy is skipped. `std::move(x)` does not move anything by itself - it is a cast to rvalue reference that permits a move constructor/assignment to be selected.

The practical rules:

- Moves happen implicitly on **prvalues** (the result of a function returning
 - The classic bug: `return std::move(local);` - the `std::move` **prevents**
 - A moved-from object must not be used, only assigned to or destroyed.
 - The rule of zero/five: if you hand-write a destructor, copy, or move for a
-

[Medium] Q. `virtual` - what is the runtime cost, and what are the two traps?

Answer. A class with virtual functions gets a **vtable** (one per dynamic type, shared by all its instances) and each object carries a **vp**tr; a virtual call reads the vptr and dispatches through the table - one extra indirection, and the call usually cannot be inlined. The cost is real but small; the design cost is that the type's behaviour is no longer knowable from the static type.

The two traps (both demonstrated in `examples/cpp/ooop/virtual_functions_and_the_vtable.cpp`):

- **Non-virtual destructor:** deleting a derived object through a base pointer
- **Hiding:** a non-virtual member with the same name in a derived class

The follow-up this sets up: `virtual` on the first virtual call also cannot be inlined, which is why hot single-type code sometimes uses the CRTP idiom (static dispatch through templates) to buy the call back.

[Hard] Q. Rule of zero, rule of three, rule of five - and how does a modern

codebase actually avoid all of them?

Answer. If a class owns a resource it must manage its five special members consistently: destructor, copy constructor, copy assignment, move constructor, move assignment. **Rule of three:** if you need one of the first three, you probably need all three. **Rule of five:** with move semantics, the count is five. **Rule of zero:** the best position is to need none - hold resources in types that already manage themselves (`unique_ptr`, `string`, `vector`) and let the implicit special members do the right thing.

Why this is the senior-level answer:

- Each hand-written special member is a place to get the invariant wrong -
- The modern idiom: `**own` with smart pointers and containers, inherit
- The escape hatch: when you genuinely need custom logic (a refcounted handle,

See also `examples/cpp/raii/raii_and_move.cpp` and `examples/cpp/smart-pointers/`.

C# Interview Questions

[Easy] Q. Value types vs reference types - and where does each one live?

Answer. Value types (`int`, `struct`, `enum`) are stored **in place** - on the stack when local, inline inside the object that contains them - and are copied wholesale on assignment. Reference types (`class`, `string`, `delegate`, arrays) store a reference; the object lives on the **managed heap** and assignment copies the reference, not the object.

The follow-ups that separate a definition from understanding:

- `string` is a reference type that behaves like a value type: it is
- A `struct` that holds a `string` or another class is a value type with
- The interop cost: a `struct` boxed to `object` (or put in an

See `examples/csharp/basics/value_vs_reference_types.cs`.

[Easy] Q. `==` vs `.Equals()` vs `ReferenceEquals` - when do they disagree?

Answer. `ReferenceEquals(a, b)` (and `object.ReferenceEquals`) asks "same object on the heap?" `a == b` is an operator: for most reference types it **calls** `Equals`, but `string` and types that overload `==` decide otherwise. `a.Equals(b)` is the virtual value-equality method.

The disagreements that actually bite:

- `string: "a" == "b"` compares content (the `==` operator for `string`)
 - `null: null == null` is true; `null.Equals(x)` throws
 - **Overloaded `==` without `Equals`:** a type that overloads `==` but not
 - Arrays: `==` on arrays is reference equality; element-wise comparison is
-

[Medium] Q. `async/await` - what does `await` actually do, and what is the

`async void` trap?

Answer. `await` does not block a thread. It registers the rest of the method as a continuation to run when the awaited task completes, and **returns from the method now** (the method becomes a state machine). The thread goes back to the pool; the continuation runs later, on the captured `synchronization context` (in UI code, back on the UI thread).

The traps:

- `async void` is for event handlers only. There is no `Task` to await, so
 - `.Result / .Wait()` on a task in a UI context deadlock: the thread
 - `async` **does not mean parallel**. It means asynchronous - overlapping
 - The `ConfigureAwait(false)` nuance: in library code you often don't need the
-

[Medium] Q. `interface` vs `abstract class` - and what did C# 8 default

interface members change?

Answer. An **interface** is a capability contract: no state, and (before C# 8) no implementation. A class can implement any number of them. An **abstract class** is a common base: it can carry state and shared behaviour, but a class has exactly **one** base class, so it competes with your real inheritance chain. The decision: repeated state/behaviour you want written once -> abstract base; a capability that unrelated types should share -> interface.

What C# 8 changed: interfaces can now have **default implementations**. That softens the line - an interface can carry behaviour - but it does not remove the distinction, because a default interface method still cannot touch instance state (there is none) and, more importantly, the reason to prefer an interface is the **multiple-implementation** freedom an abstract base cannot give. Default members are for extending an existing interface without breaking implementers (add a method with a sensible default), not for replacing a base class.

See `examples/csharp/oop/interface_vs_abstract_class.cs`.

[Hard] Q. What is the difference between `Task`, `ValueTask`, and

`Task<T>` - and when is each the right return type?

Answer. `Task<T>` is the standard async result: a heap-allocated object representing a computation that yields `T`. `ValueTask<T>` is the **low-allocation** alternative for methods that usually complete synchronously: it can either wrap an already-available `T` (no allocation) or a `Task<T>` (allocation happens only on the async path). `Task` (no `T`) is the fire-and-complete-notify version for work with no result.

The trade-off, stated precisely:

- **Default to** `Task<T>`. It is composable, can be stored, awaited multiple
- **Use** `ValueTask<T>` **only on hot paths** where the method frequently returns
- The interview-grade framing: `ValueTask` is an optimization with a contract

See `examples/csharp/async/` for the `Task.WhenAll` and cancellation patterns.

Go

Go interviews focus on the concurrency model (goroutines + channels), explicit error handling, and the small set of language rules that differ from C-family intuition (no inheritance, no exceptions, `interface{}` everywhere). (Snippets below validated by inspection - no Go toolchain on the authoring machine.)

[Easy] Q. What is a goroutine, and how is it different from a thread?

Answer. A goroutine is a function executed concurrently by the Go **runtime**, not the OS. The runtime multiplexes thousands of them onto a small pool of OS threads (the GMP model: Goroutines, runnables on Machine threads, driven by Ps). A goroutine starts at ~2KB of stack that grows on demand, versus ~1-8MB fixed for an OS thread - so spawning them is cheap, and the idiom is one goroutine per connection/task rather than a thread pool you size by hand.

```
package main

import "fmt"

func worker(id int, out chan<- int) {
    out <- id * id // blocks until the value is received
}

func main() {
    out := make(chan int, 3) // buffered: workers never block
    for i := 1; i <= 3; i++ {
        go worker(i, out) // start a goroutine
    }
    for i := 1; i <= 3; i++ {
        fmt.Println(<-out) // 1, 4, 9 - in some order
    }
}
```

The key mental shift: concurrency is **explicit dataflow**. Goroutines communicate by sending/receiving on channels, and a send on an unbuffered channel blocks until the other side is ready - that blocking is the synchronisation, not a lock.

[Medium] Q. Buffered vs unbuffered channels - when does each cause a goroutine to block?

Answer. An **unbuffered** channel is a direct handoff: send blocks until a receiver is waiting, and receive blocks until a sender is waiting. A **buffered** channel of size N absorbs up to N values without a waiting peer; send blocks only when the buffer is full, receive only when it is empty.

```
ch := make(chan int) // unbuffered
ch <- 1               // BLOCKS forever here: no receiver yet

bch := make(chan int, 2) // buffered, capacity 2
bch <- 1                 // fine: goes into the buffer
bch <- 2                 // fine: buffer full now
// bch <- 3             // would BLOCK: buffer full, no receiver
```

The classic deadlock is a send on an unbuffered channel (or a full buffer) with nobody receiving - the runtime kills the program when every goroutine is blocked. Rule of thumb: use unbuffered channels when you want strict handoff/synchronisation, buffered channels when you want to decouple producer and consumer rates or absorb bursts.

[Medium] Q. How does Go handle errors, and why no exceptions?

Answer. Errors are **values**: fallible functions return an extra `error` (an interface), and the caller checks it explicitly, usually immediately. There are no exceptions, so there is no hidden control flow and no "this call might throw halfway through" reasoning - every failure point is visible at the call site.

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}

func main() {
    r, err := divide(10, 2)
    if err != nil {
        fmt.Println("error:", err) // handle the failure explicitly
        return
    }
    fmt.Println(r) // 5
}
```

The standard library's `errors.Is / errors.As` and the `%w` wrap verb let you **wrap** errors to add context while keeping the original checkable: `fmt.Errorf("reading config: %w", err)` means `errors.Is(wrapped, os.ErrNotExist)` still matches the underlying `os.ErrNotExist`. The trade-off to state: verbosity (the `if err != nil` dance) in exchange for explicitness - and the idiomatic fix for deep chains is to let `?-style` helpers or a single early return at each boundary do the work.

[Medium] Q. What are interfaces in Go, and why is `io.Reader` so central?

Answer. An interface is a set of method requirements, and a type satisfies it **implicitly** by implementing the methods - no `implements` keyword. This makes interfaces small and adopted at the point of use: a function takes `io.Reader` (a single `Read([]byte) (int, error)`) and can be handed a file, a network connection, a `bytes.Buffer`, or your own type, with no changes.

```
// Anything with Read(p []byte) (int, error) is an io.Reader.
type MySource struct{ data []byte; pos int }

func (m *MySource) Read(p []byte) (int, error) {
    if m.pos >= len(m.data) {
        return 0, io.EOF
    }
    n := copy(p, m.data[m.pos:])
    m.pos += n
    return n, nil
}

// Now this generic decoder works on files, HTTP bodies, AND MySource:
func decode(r io.Reader) ([]byte, error) { return io.ReadAll(r) }
```

The design principle: **accept interfaces, return concrete types**. Small interfaces (1-3 methods) keep things **composable** - `io.Reader`, `io.Writer`, `io.Closer` combine into `io.ReadWriter`, `io.ReadCloser`, etc. The flip side to mention: implicit satisfaction can make it hard to find what implements an interface, and empty `interface{}` / `any` erases type safety until you assert.

[Hard] Q. A goroutine leaks in your service. What does that mean, how do

you find it, and how do you fix it?

Answer. A **goroutine leak** is a goroutine that can never finish - usually blocked forever on a channel send/receive, a lock, or a `select` with no ready case and no exit path. Each one holds its stack and any closures (and the data they reference), so a leak is a slow memory growth plus lost work, and it shows up as "goroutine count climbs linearly with requests" in a runtime profile.

How you find it: the runtime traceback (`/debug/pprof/goroutine?debug=1` or `go tool pprof`) groups leaked goroutines by the stack they're blocked on - you'll see hundreds parked in `<-ch` or `select`. The fixes, in order of idiom:

- 1 **Context cancellation** - thread a `context.Context` into long-running
- 2 **Ensure every channel send has a receiver** (or a buffer that will be
- 3 **Prefer `select` with an exit case** over a bare `<-ch`, and close

The answer they're looking for is the mental model: a goroutine is only as dead as its last blocking operation, so "leak" almost always means "someone stopped receiving (or stopped cancelling) and this goroutine is waiting on it forever."

Java Interview Questions

[Easy] Q. What is the difference between JDK, JRE and JVM?

Answer. **JVM** executes bytecode and is the thing that makes Java portable - one `.class` file runs anywhere a JVM exists. **JRE** is the JVM plus the standard class libraries: enough to run Java. **JDK** is the JRE plus development tools (`javac`, `jdb`, `jar`): enough to build Java.

"Write once, run anywhere" is a property of bytecode plus a per-platform JVM, not of the source language.

[Easy] Q. Primitive vs reference types?

Answer. Primitives (`int`, `double`, `boolean`, `char`, ...) hold values directly, live on the stack when local, cannot be null, and have no methods. Reference types hold a pointer to a heap object, can be null, and carry methods.

The consequence that produces bugs:

```
String a = new String("hi");
String b = new String("hi");
a == b           // false - different objects
a.equals(b)      // true  - same characters
```

`==` compares references for objects, values for primitives. **Always use `.equals()` for objects.**

Autoboxing makes this worse:

```
Integer x = 127, y = 127;
x == y;           // true  - cached (-128..127)
Integer p = 128, q = 128;
p == q;           // false - outside the cache
```

Same code, different answer depending on the value. Never compare boxed types with `==`.

[Hard] Q. Explain the equals/hashCode contract.

Answer. If `a.equals(b)` then `a.hashCode() == b.hashCode()`, **always**. The reverse is not required - equal hash codes with unequal objects is just a collision.

Override one and you must override the other. Break it and hash-based collections fail silently:

```
Map<Point, String> map = new HashMap<>();
map.put(new Point(1, 2), "origin-ish");
map.get(new Point(1, 2)); // null, if hashCode() was not overridden
```

The lookup hashes to a different bucket and never finds the entry.

The second half of the contract: **a key's hash must not change while it is in a collection**. Mutating a field used by `hashCode` strands the entry - present but unreachable, and `remove()` will not find it either. Prefer immutable keys; `record` types give you correct `equals/hashCode` for free.

[Medium] Q. Compare the collections.

Answer.

Interface	Implementation	Ordering	Notes
List	ArrayList	insertion	O(1) index, O(n) middle insert. The default.

	LinkedList	insertion	O(1) ends, poor locality - rarely the right choice
Set	HashSet	none	O(1) average
	LinkedHashSet	insertion	O(1), remembers order
	TreeSet	sorted	O(log n), needs Comparable/Comparator
Map	HashMap	none	O(1) average; chains become trees past 8 entries
	LinkedHashMap	insertion or access	access-order mode gives you an LRU cache
	TreeMap	sorted	O(log n), range queries

`ArrayList` unless you need ordering or sorting. `LinkedList` is almost never the answer despite what its Big-O table suggests - cache locality wins.

[Medium] Q. Checked vs unchecked exceptions?

Answer. **Checked** (extend `Exception`) must be declared or caught - the compiler enforces it. Intended for recoverable conditions: `IOException`, `SQLException`.

Unchecked (extend `RuntimeException`) need no declaration. Intended for programming errors: `NullPointerException`, `IllegalArgumentException`, `IndexOutOfBoundsException`.

`Error` (e.g. `OutOfMemoryError`, `StackOverflowError`) signals a JVM-level problem you should not catch.

The design debate is worth knowing: checked exceptions were meant to force handling, but in practice they encourage `catch (Exception e) {}` and leak implementation details through signatures. Later languages (C#, Kotlin, Scala) deliberately omitted them. Worked example:

`examples/java/exceptions/exception_handling_basics.java`.

[Hard] Q. What is type erasure?

Answer. Generics are a **compile-time** feature. The compiler checks types, then erases them - `List<String>` becomes plain `List` in the bytecode, with casts inserted automatically. This was done for backwards compatibility with pre-generics code.

The consequences you can be asked to explain:

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
a.getClass() == b.getClass();      // true - both are just ArrayList

// Illegal, because the type is not available at runtime:
// if (obj instanceof List<String>) { }
// T[] array = new T[10];
```

You also cannot overload on `List<String>` vs `List<Integer>` - after erasure both methods have identical signatures. Contrast with C#, where generics are reified and the runtime does know the type argument.

[Medium] Q. How does garbage collection work?

Answer. The JVM reclaims objects no longer **reachable** from GC roots (stack references, statics, JNI references). Note it is reachability, not reference counting - so cycles are collected correctly.

The design rests on the **generational hypothesis**: most objects die young. So the heap is split:

- **Young generation** (Eden + two survivor spaces) - collected frequently with
- **Old generation** - objects that survived several collections; collected

Modern collectors (G1, ZGC, Shenandoah) do most work concurrently to keep pause times low - ZGC targets sub-millisecond pauses on large heaps.

`System.gc()` is a suggestion the JVM may ignore. You cannot force collection, and calling it is almost always a mistake. Memory leaks in Java are real but take a different form: objects still referenced but never used - a static collection that only ever grows, an unremoved listener, an unclosed resource.

[Medium] Q. What does the Stream API give you?

Answer. A declarative pipeline over a sequence, with lazy intermediate operations and one terminal operation that triggers execution.

```
List<String> names = people.stream()
    .filter(p -> p.age() >= 18)           // intermediate, lazy
    .map(Person::name)                   // intermediate, lazy
    .sorted()
    .toList();                           // terminal, runs the pipeline
```

Nothing executes until the terminal operation, which allows fusion and short-circuiting (`findFirst` stops early).

Two cautions worth raising unprompted: a stream is **single-use** - reusing one throws

`IllegalStateException`; and `parallelStream()` is not free speed. It uses the common `ForkJoinPool`, so it helps only for large datasets with CPU-bound, independent, side-effect-free work, and can be slower otherwise. Worked example: `examples/java/streams/group_and_collect_with_streams.java`.

[Medium] Q. `String`, `StringBuilder`, `StringBuffer`?

Answer. `String` is **immutable** - every concatenation allocates a new object, so building in a loop is $O(n^2)$.

`StringBuilder` is a mutable buffer, not thread-safe, and is what you want. `StringBuffer` is the synchronised version: slower, and almost never needed since a builder is normally local to one thread.

Note the compiler rewrites simple + concatenation into builder calls, so `"a" + b + "c"` is fine - it is concatenation **inside a loop** that is the problem. Worked example:

`examples/java/strings/string_immutability_and_stringbuilder.java`.

JavaScript Interview Questions

[Easy] Q. `var`, `let` and `const`?

Answer.

	Scope	Hoisting	Reassign
<code>var</code>	function	hoisted, initialised undefined	yes
<code>let</code>	block	hoisted into the temporal dead zone	yes
<code>const</code>	block	same as <code>let</code>	no

`const` prevents **rebinding the name**, not mutation of the value:

```
const user = { name: "Sara" };
user.name = "Ali";           // fine - the object is mutable
user = {};                   // TypeError - cannot rebound
```

Use `const` by default, `let` when you must reassign, and `var` never.

[Medium] Q. What is a closure?

Answer. A function that retains access to the variables of the scope it was created in, even after that scope has returned.

```
function counter() {
  let count = 0;           // stays alive via the closure
  return {
    increment: () => ++count,
    value: () => count,
  };
}
const c = counter();
c.increment(); c.increment();
c.value();           // 2
```

`count` is genuinely private - unreachable except through the returned functions. This is the basis of the module pattern, memoisation, and every callback that "remembers" something.

The classic interview trap:

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3, 3, 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 0, 1, 2
```

`var` is function-scoped, so all three callbacks close over **one** variable, which is 3 by the time they run. `let` creates a fresh binding per iteration. Worked example: [examples/javascript/basics/closures.js](https://github.com/InterviewCracker/javascript-examples/blob/master/javascript/basics/closures.js).

[Hard] Q. Explain the event loop.

Answer. JavaScript is single-threaded but non-blocking. The runtime has:

- a **call stack** - what is executing now;
- **Web APIs / host APIs** - timers, network, file I/O, running outside the
- a **macrotask queue** - `setTimeout`, I/O callbacks, UI events;
- a **microtask queue** - promise callbacks, `queueMicrotask`.

The loop: when the stack empties, **drain the entire microtask queue**, then take **one** macrotask, then drain microtasks again. Microtasks always have priority.

```
console.log("1");
setTimeout(() => console.log("2"), 0); // macrotask
Promise.resolve().then(() => console.log("3")); // microtask
console.log("4");
// 1, 4, 3, 2
```

2 comes last despite a 0 ms delay, because the microtask queue is drained first. The follow-up: an infinite chain of microtasks **starves** the macrotask queue and freezes the page - microtask priority is not free. Worked example: `examples/javascript/async/promises_and_event_loop.js`.

[Medium] Q. How does `this` get its value?

Answer. In a normal function `this` is determined by **how it is called**, not where it is defined. Four rules, in precedence order:

- 1 **new** - `this` is the new object.
- 2 **Explicit** - `call`, `apply`, `bind` set it.
- 3 **Implicit** - `obj.method()` sets `this` to `obj`.
- 4 **Default** - `undefined` in strict mode, `globalThis` otherwise.

Arrow functions ignore all four: they have **no** `this` **of their own** and inherit it lexically from the enclosing scope. That is the fix for the classic bug:

```
class Timer {
  constructor() { this.seconds = 0; }
  startBroken() { setInterval(function () { this.seconds++; }, 1000); } // wrong `this`
  start()       { setInterval(() => { this.seconds++; }, 1000); }      // correct
}
```

Worked example: `examples/javascript/basics/this_and_binding.js`.

[Medium] Q. `==` vs `===`?

Answer. `===` compares type and value with no conversion. `==` coerces first, using rules that are genuinely surprising:

```
0 == ""; // true
0 == "0"; // true
"" == "0"; // false <- not transitive!
null == undefined // true
null == 0 // false
NaN == NaN // false
```

Always use `===`. The one accepted exception is `x == null`, which checks for `null` or `undefined` in a single test.

For `NaN`, use `Number.isNaN(x)`. `NaN !== NaN` is required by IEEE 754 and is also why `[NaN].includes(NaN)` is true (it uses `SameValueZero`) while `[NaN].indexOf(NaN)` is `-1` (it uses `===`).

[Medium] Q. What is prototypal inheritance?

Answer. Every object has a hidden link to another object, its **prototype**. A property miss walks up that chain until it is found or the chain ends at `null`.

```
const animal = { speak() { return `${this.name} makes a sound`; } };
const dog = Object.create(animal);
dog.name = "Rex";
dog.speak(); // found on the prototype
```

`class` syntax is sugar over this - there are no classical classes underneath, just constructor functions and prototype objects. That is why you can add a method to every existing instance at runtime by assigning to the prototype, which classical inheritance does not allow.

[Medium] Q. Promise, async/await, and how do you run things in parallel?

Answer. A promise represents a value that is not ready yet, in one of three states: pending, fulfilled, rejected. `async/await` is syntax over promises - an `async` function always returns one, and `await` suspends until it settles.

The mistake that matters:

```
// Sequential - 3 seconds, and usually a bug
const a = await fetchA();
const b = await fetchB();

// Parallel - 1 second
const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

If the calls are independent, awaiting them one at a time wastes the whole latency of each. Know the four combinators:

- `Promise.all` - all succeed, or reject on the **first** failure.
- `Promise.allSettled` - wait for all, never rejects; inspect each result.
- `Promise.race` - first to **settle**, success or failure (used for timeouts).
- `Promise.any` - first to **succeed**.

Worked example: `examples/typescript/async/async_await_parallel_vs_sequential.ts`.

[Medium] Q. Debounce vs throttle?

Answer. Both limit how often a function runs, in opposite ways.

- **Debounce** - wait until the events stop, then run once. Search-as-you-type,
- **Throttle** - run at most once per interval while events continue. Scroll

"Fire after the user stops typing" is debounce; "fire at most every 200 ms while scrolling" is throttle. Worked example: `examples/javascript/utilities/debounce_with_cancellation.js`.

Python Interview Questions

[Easy] Q. Mutable vs immutable - and why does it matter?

Answer. **Immutable:** int, float, str, tuple, frozenset, bytes. **Mutable:** list, dict, set, bytearray, most custom classes.

It decides three things: whether a value can be a dict key (hashable, which requires immutable), whether passing it to a function can change the caller's object, and whether it is safe to share between threads.

The classic bug it causes:

```
def append_to(item, target=[]):      # DANGER: evaluated once, at def time
    target.append(item)
    return target

append_to(1)    # [1]
append_to(2)    # [1, 2] <- the same list, still there
```

Default arguments are evaluated **once**, when the function is defined. Use `None` as the sentinel:

```
def append_to(item, target=None):
    if target is None:
        target = []
    target.append(item)
    return target
```

[Easy] Q. is vs ==?

Answer. `==` compares **values** (via `__eq__`); `is` compares **identity** - whether both names point to the same object.

```
a = [1, 2, 3]
b = [1, 2, 3]
a == b    # True - same contents
a is b    # False - two distinct objects
```

Use `is` only for singletons: `is None`, `is True`, `is False`.

The confusing part is small-integer and string caching. `256 is 256` is `True` because CPython interns small integers (-5 to 256); `1000 is 1000` may be `False` depending on context. This is an implementation detail you must never rely on - and a favourite trick question.

[Hard] Q. What is the GIL and what does it mean for you?

Answer. The Global Interpreter Lock is a mutex ensuring only one thread executes Python bytecode at a time, even on a many-core machine. It exists because CPython's reference-counting memory management is not thread-safe, and one coarse lock was far simpler and faster for single-threaded code than locking every object.

The practical consequence:

- **CPU-bound work gets no speedup from threads.** Four threads on four cores
- **I/O-bound work does benefit**, because the GIL is released around blocking

Worth knowing this is changing: PEP 703 adds an optional free-threaded build (3.13+) with no GIL, and 3.12+ supports per-interpreter GILs. Saying "the GIL is being removed as an option, gradually" is more accurate than either "it's gone" or "Python can't do parallelism".

[Medium] Q. What is a generator and why use one?

Answer. A function containing `yield`. Calling it returns a lazy iterator that computes values on demand rather than building a list.

```
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()          # one line in memory, not the whole file
```

Two benefits: **constant memory** regardless of input size, and the ability to represent infinite sequences. The costs: you can only iterate once, and you cannot index or take `len()`.

Use `()` instead of `[]` to get a generator expression: `sum(x*x for x in range(10**8))` never allocates the list. Worked example: `examples/python/generators/log_pipeline_with_generators.py`.

[Medium] Q. What is a decorator?

Answer. A callable that takes a function and returns a replacement, used to add behaviour without editing the original.

```
import functools, time

def timed(func):
    @functools.wraps(func)          # preserve __name__, __doc__, signature
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__} took {time.perf_counter() - start:.4f}s")
    return wrapper

@timed
def work(): ...

@timed is exactly work = timed(work).
```

`functools.wraps` **is the detail interviewers check for**. Without it the decorated function reports the wrapper's name and loses its docstring, which breaks introspection, documentation tools and debugging. Worked example: `examples/python/decorators/retry_decorator_with_backoff.py`.

[Medium] Q. Shallow vs deep copy?

Answer. A **shallow copy** (`list(x)`, `x[:]`, `copy.copy`) makes a new outer object holding **references to the same inner objects**. A **deep copy** (`copy.deepcopy`) recursively copies everything.

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
shallow[0].append(99)
original          # [[1, 2, 99], [3, 4]]  <- mutated!

deep = copy.deepcopy(original)
deep[0].append(100)
original          # unchanged
```

Deep copy is slower and fails or loops on some structures - it handles cycles, but not open file handles or sockets. Prefer immutable data over copying.

[Medium] Q. What does a context manager do?

Answer. Guarantees cleanup via `__enter__` / `__exit__`, so a resource is released even if an exception is raised.

```
from contextlib import contextmanager

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield
    finally:
        # runs on exception too
        print(f"{label}: {time.perf_counter() - start:.3f}s")
```

`with open(...)` is the everyday example: the file closes on any exit path. Worked example: `examples/python/context-managers/custom_context_managers.py`.

[Medium] Q. `__str__` vs `__repr__`?

Answer. `__repr__` is for **developers** - unambiguous, ideally valid Python that would recreate the object. It is what the REPL and containers show. `__str__` is for **users** - readable.

If you implement only one, implement `__repr__`: `__str__` falls back to it, but not the other way round. Worked example: `examples/python/oop/dunder_methods.py`.

[Medium] Q. What are `*args` and `**kwargs`?

Answer. `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict. In a call, the same symbols unpack:

```
def f(a, b, c): ...
values = [1, 2, 3]
f(*values)                # unpacks positionally

config = {"a": 1, "b": 2, "c": 3}
f(**config)               # unpacks by keyword
```

This is what lets a decorator wrap any function signature.

Rust

Rust interviews test one idea above all: do you understand **ownership**, and can you reason about when the borrow checker accepts or rejects code? The language features (traits, enums, iterators) are the icing; memory safety without a garbage collector is the substance.

[Easy] Q. What is ownership, and what does it mean to "move" a value?

Answer. Every value has exactly one **owner**; when the owner goes out of scope, the value is dropped.

Assignment and function arguments **move** the value by default - the original binding is no longer usable. This is what guarantees no double-free and no use-after-free without a GC.

```
fn takes_ownership(x: String) -> u32 {
    x.len() as u32
}

fn main() {
    let s = String::from("hello");
    let n = takes_ownership(s);    // `s` is MOVED into the function
    assert_eq!(n, 5);
    // println!("{}", s);          // COMPILER ERROR: `s` was moved
}
```

Types that implement `Copy` (`ints`, `bool`, `floats`, `char`, and `fixed-size arrays` of those) are copied instead of moved, which is why `let y = x;` feels free for `i32` but not for `String`.

[Medium] Q. What are borrowing and lifetimes, and when does the borrow

checker reject your code?

Answer. A **borrow** (`&T` / `&mut T`) lets you use a value without taking ownership. The rules: any number of shared borrows, OR exactly one mutable borrow - never both at once. A **lifetime** annotation tells the compiler how long a reference may outlive the data it points to; you rarely write them explicitly, but `longest<'a>(a: &'a str, b: &'a str) -> &'a str` shows the rule that the returned reference may live no longer than the shorter-lived input.

```
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}

fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or("")
}

fn main() {
    let s1 = String::from("long string is long");
    let s2 = String::from("xyz");
    let result = longest(s1.as_str(), s2.as_str());
    assert_eq!(result, "long string is long");
    assert_eq!(first_word(" hello world"), "hello");
}
```

The classic rejection, and the one to be able to explain out loud:

```
fn main() {
    let result;
    {
        let s = String::from("hello");
        result = longest(s.as_str(), "world");
    }                                // `s` dropped here
    let _ = result.len();           // ERROR: `s` does not live long enough
}
```

```
}
```

`result` borrows from `s`, which dies at the closing brace - returning a reference to it would dangle. The fix is to return an owned `String` (`.to_string()`) or keep `s` alive for as long as `result` is used.

[Medium] Q. How does Rust handle errors - `Option`, `Result`, and the `?`

operator?

Answer. Failure is a **value**, not an exception. `Option<T>` is `Some(T)` or `None` for "no value"; `Result<T, E>` is `Ok(T)` or `Err(E)` for recoverable failures. The `?` operator propagates an error: on `Err` it returns early from the current function, on `Ok` it unwraps the value - replacing the nested-matching pyramid of code.

```
fn parse_int(s: &str) -> Result<i64, String> {
    s.parse::<i64>().map_err(|e| format!("not a number: {e}"))
}

fn safe_div(a: i64, b: i64) -> Result<i64, String> {
    if b == 0 { return Err("division by zero".to_string()); }
    Ok(a / b)
}

// `?` returns early on Err, unwraps on Ok:
fn process(input: &str) -> Result<i64, String> {
    let n = parse_int(input)?;
    let half = safe_div(n, 2)?;
    Ok(half)
}

fn main() {
    assert_eq!(process("10"), Ok(5));
    assert!(process("abc").is_err());
    assert_eq!(safe_div(1, 0), Err("division by zero".to_string()));
}
```

The trade-off to state: `?` makes pipelines linear but forces every function to thread a `Result` through its signature - errors are explicit at every boundary (a feature), whereas exception-based languages let failures surface only where they are caught (and silently, if never caught).

[Medium] Q. When do you reach for `Box`, `Rc`, or `RefCell`?

Answer. They solve the three limitations of the stack-ownership model:

- `Box<T>` - heap allocation / unknown-at-compile-time size /
- `Rc<T>` - shared ownership: many owners, one data. Reference
- `RefCell<T>` - interior mutability: immutable binding, mutable

```
use std::rc::Rc;
use std::cell::RefCell;

struct Node {
    value: i32,
    next: RefCell<Option<Rc<Node>>>,
}

fn push(head: &Rc<RefCell<Option<Rc<Node>>>>, value: i32) {
    let new = Rc::new(Node { value, next: RefCell::new(None) });
    (*head).borrow_mut().replace(new); // unique write access for a moment
}

fn len(head: &Rc<RefCell<Option<Rc<Node>>>>) -> usize {
```

```

    let mut n = 0;
    let mut cur = head.borrow().clone(); // clone the Rc (cheap refcount bump)
    while let Some(node) = cur {
        n += 1;
        cur = node.next.borrow().clone();
    }
    n
}

```

`Rc` outside + `RefCell` inside is the standard shape for data with multiple owners AND runtime-rearranged structure (graphs, trees you mutate in place). The cost: `borrow()/borrow_mut()` panic at runtime on rule violations, and `Rc` increments/decrements a counter on every clone/drop.

[Hard] Q. Walk through what happens when a function with an `Option`

return value is called, and why `unwrap()` is a code smell in production.

Answer. The caller **must** handle both cases - the type system makes `None` a visible branch, usually via `match`, `if let`, or `? .unwrap()` (and `expect()`) say "I guarantee this is `Some`, panic otherwise" - fine in tests and prototypes, but in production it converts a handleable absence into a crash, typically deep in a request path where the real cause (a bad input, a missing config key) is far away. The production pattern is to propagate: return a `Result/Option` upward with `?`, map the `None` into a meaningful error with `ok_or(...)`, or branch deliberately with a default only when a default is genuinely correct.

```

fn find_user(id: u64) -> Option<String> {
    (id == 1).then(|| String::from("ada"))
}

fn greet(id: u64) -> Result<String, String> {
    let user = find_user(id)
    .ok_or_else(|| format!("no user with id {id}"))?;
    Ok(format!("hello {user}"))
}

fn main() {
    assert_eq!(greet(1), Ok("hello ada".to_string()));
    assert!(greet(2).is_err());
}

```

The follow-up: `Option` and `Result` are enums with **zero overhead** in the `Some/Ok` case (a niche optimization means `Option<&T>` is the same size as `&T`), so wrapping fallible values costs nothing at runtime - the cost is purely at the call sites, which is exactly where you want the attention.